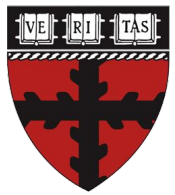
Lecture 14: Serverless Deployment
Functions, services, identity, and jobs

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Announcements

Upcoming dates

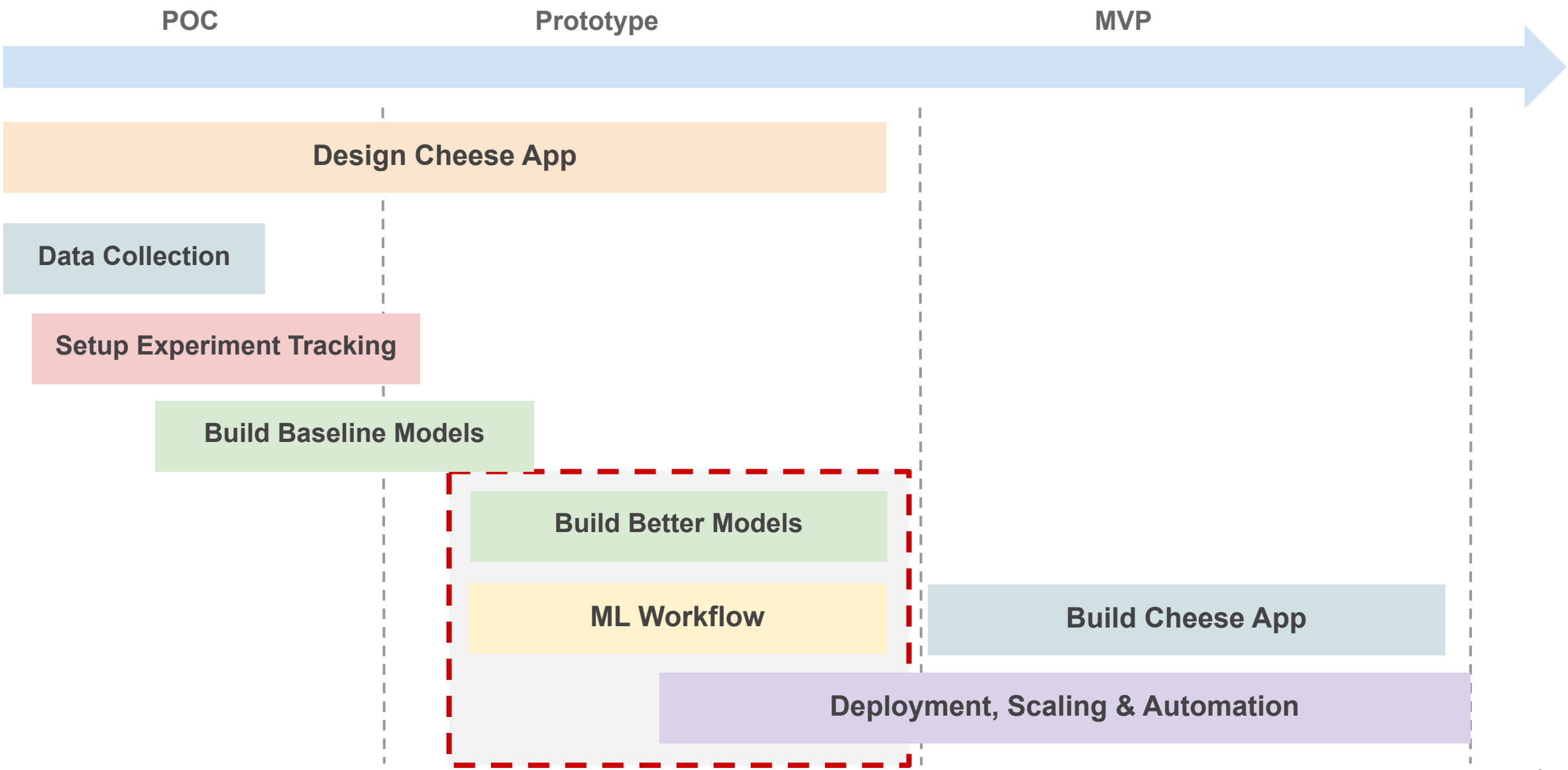
- Quiz 4: November 3
- Milestone 3: due November 12
- Today: deployment tools for your project
- Submission details and deadlines: Canvas

Today’s tutorial

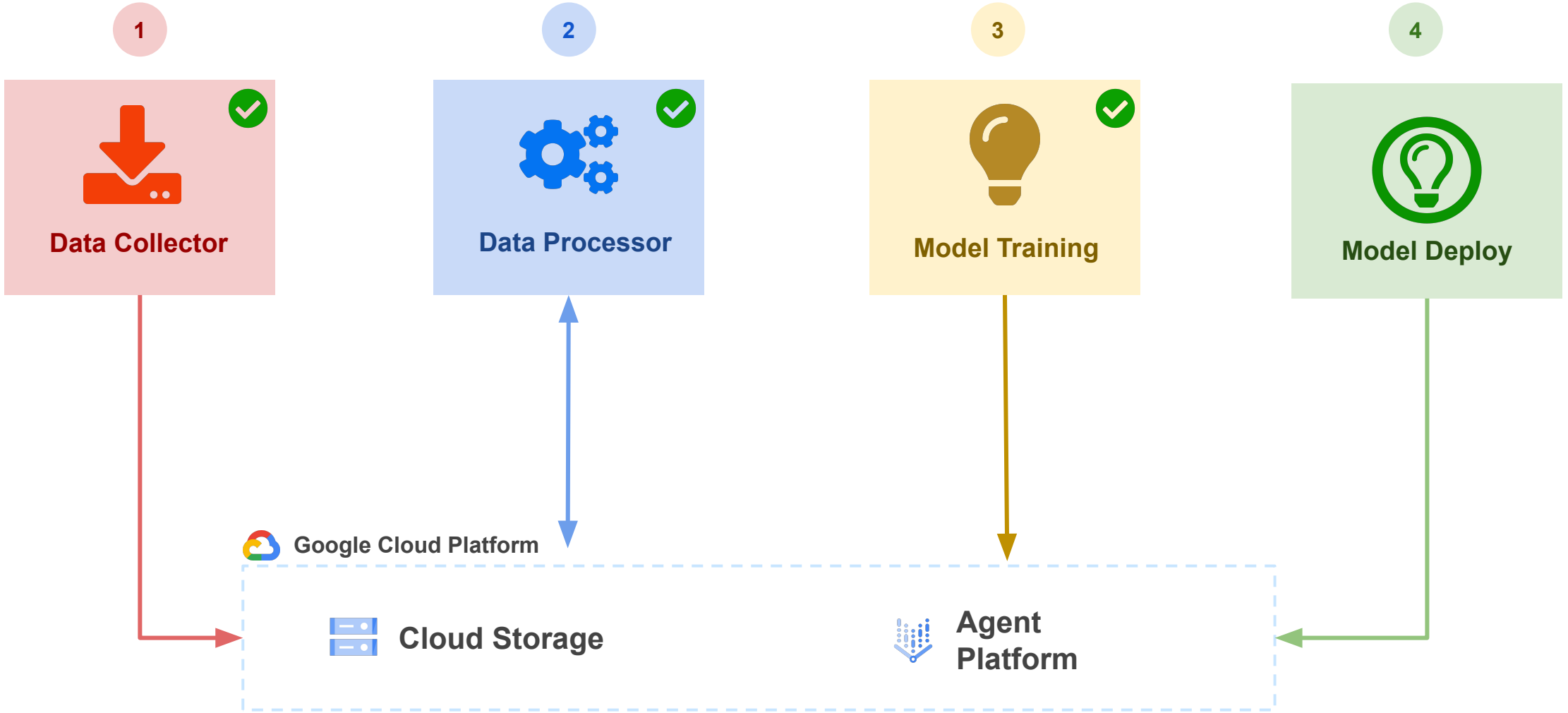
Four steps, one translation application

1. Function: code → public endpoint
2. Container: local web UI → public service
3. Identity: service account → speech API
4. Job: finite work → saved text and audio

Review: Cheese App Status



Cheese App: architecture direction



Why use the translation example?

Same pattern, simpler example

- Cheese App goal: train and host our classifier
- Translation: an existing external model API
- Today: deploy the application that calls the API
- Cheese-model serving comes later

Our service and the external API

Our service connects user and model

- User → our Cloud Run service → translation API
- Our service returns the translation to the user
- Backend: server-side; can expose an API and call services
- Code or container → deploy → test

Serverless

What the platform manages

- Runs code on managed infrastructure
- Starts and scales instances as needed
- You configure dependencies, resources, and identity
- Cost depends on billing and scaling choices

Services and jobs

Two ways to execute a container

- Service: handles HTTP requests at a stable URL
- Instances may scale to zero between requests
- Job: starts when executed, then finishes
- Identity and permissions apply to either

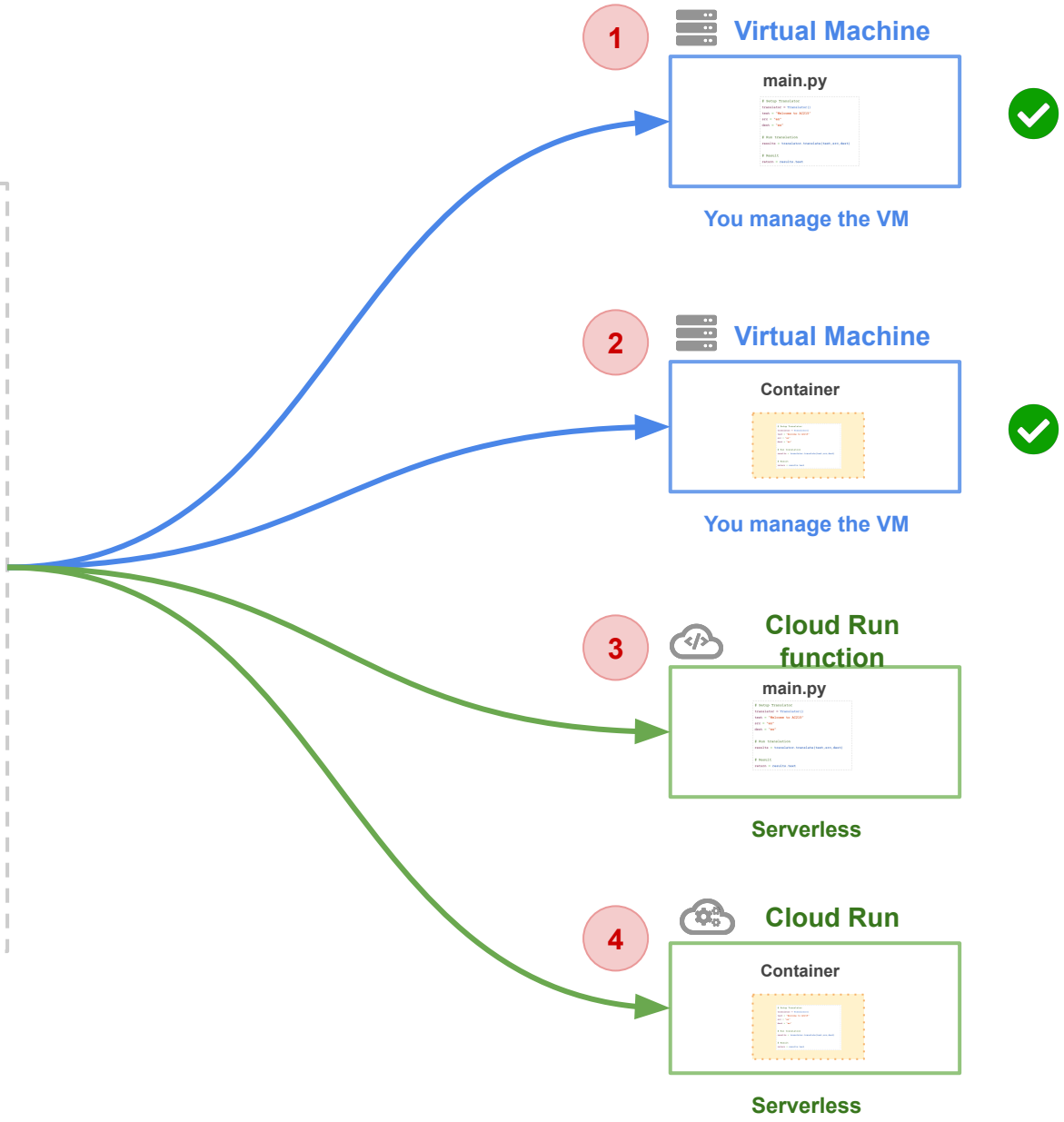
Deployment Options

Simple Translate App

```
from googletrans import Translator

text = "Welcome to AC215"
src = "en"
dest = "es"

result = Translator().translate(
    text, src=src, dest=dest
)
translated_text = result.text
```



Cloud Run functions

Source code → managed service

- You supply a handler + requirements.txt
- Google builds and stores the container image
- Functions Framework calls your HTTP handler
- Cloud Run serves requests and scales instances

Part 1: Deploy a function

Source code → public endpoint

- Cloud Run → Services → Write a function
- Python 3.12 + main.py + requirements.txt
- Deploy, vary the query, and inspect Logs
- Open Part 1 in the tutorial

Cloud Run: services and jobs

Deploy your existing application

- Publish the tested image to a registry
- Service: public URL for the translation UI
- Job: finite processing that exits
- Cloud Run manages infrastructure and scaling

Part 2: Deploy the web app

Localhost → public web application

- Run the prebuilt translation UI locally
- Deploy the same public image to Cloud Run
- Test translation at the public URL
- Open Part 2 in the tutorial



Part 3: Add speech

Give our service permission to call another API

- Attach a service account to the web service
- Grant Agent Platform access; enable speech
- Return WAV audio directly to the browser
- Local development: ADC + impersonation

Part 4: Run a batch job

Run once and keep the result

- Grant job read/create access to private buckets
- Read passage → translate → generate speech
- Save text + WAV + manifest, then exit
- Run in Console; reuse with the CLI helper

Serverless trade-offs

What still needs your attention

- Cold starts: extra startup latency
- Stateless instances: save durable output in Storage
- Cloud integration: IAM, quotas, and provider APIs
- Cost: depends on billing and scaling settings